

第11章 摊还分析

建议学时:4-5 小时 | 难度:★★★★★ | 读者背景:已掌握 C 语言,DS&A 零基础

🎯 本章学习目标

- 分清三种"平均":最坏、平均、**摊还**——本章主题是第三种
- 掌握**聚集分析**(栈序列例证)与**会计方法**(动态数组"存 3 元"完整证明)
- 掌握**势能方法**:用 Φ 重推栈,理解通用框架与选 Φ 的直觉
- 会用摊还眼光看第 6 章的堆:buildHeap 为何 $O(n)$,斜堆 merge 为何摊还 $O(\log n)$
- 完成从 C 的 `realloc` 手动扩容到 C++ `vector` 自动倍增的迁移
- 能独立实测 `vector` 扩容拐点,验证"总拷贝 $\approx 2n$ "

💡 从 C 到 C++:本章主题如何迁移

本章是全书数学最重的一章,动机却非常"工程"。回忆 C 里的动态数组: `realloc` 扩容要么每次多开固定格数,要么自己"翻倍"。你凭直觉知道"翻倍更好"却说不出为什么,本章给出严格证明:倍增使 n 次插入总拷贝 $\approx 2n$,摊到每次平均 $O(1)$ 。

C++ 的 `vector` 把这套证明封装进语言: `push_back` 平均 $O(1)$ 、最坏 $O(n)$ 。C 里 `realloc` 后指针失效的坑,在 C++ 里是"迭代器失效"——同一机制的两副面孔。

数学工具不新: Σ 求和与等比级数(第 2 章)、堆与斜堆(第 6 章)。新的是**视角**:不看单个操作,看整个序列的"总账"——聚集、会计、势能三种方法,是同一本总账的三种记法。

📦 前置知识自查

数学前置:求和记号 Σ 、等差与等比级数: $\Sigma_{i=1}^n i = n(n+1)/2$, $\Sigma_{i=0}^n 2^i = 2^{n+1} - 1$;对数 $\log_2 n$ (见第 2 章)。

C 语言前置: `realloc` 手动扩容动态数组(第 3 章)、函数与循环。

依赖的前面章节:第 3 章 `vector` 倍增扩容与 `size/capacity`;第 6 章 buildHeap、斜堆 merge 与二项队列 (§11.5 的结论建立在它们结构上)。

🚀 本章学习路线

- 第一阶段(1.5 小时)**:读 §11.1–§11.2,建立"序列总账"直觉
- 第二阶段(1.5 小时)**:读 §11.3–§11.4,吃透"存 3 元"证明与势能框架
- 第三阶段(2 小时)**:读 §11.5–§11.6,把结论接到第 6 章的堆上;运行代码观察拐点,完成章末实验
- 验收标准**:能手写证明"倍增扩容总拷贝 $\approx 2n$ ",说清三种方法的区别与联系

本章知识导图

- §11.1 动机:摊还 vs 平均 vs 最坏
- §11.2 聚集分析:栈操作序列
- §11.3 会计方法:动态数组的"存3元"证明
- §11.4 势能方法:势能函数与通用框架
- §11.5 摊还在堆上的应用
- §11.6 工程印证:实测扩容拐点
- §11.7 三种方法的选用总结

§11.1 动机:摊还 vs 平均 vs 最坏

11.1.1 三种"平均"不是一回事

前几章我们反复说"vector 尾部插入平均 $O(1)$ 、最坏 $O(n)$ "。这个"平均"是什么意思?复杂度分析里有三种不同的说法,极易混为一谈:

说法	对什么取平均	需要什么假设	例子
最坏情形	不取平均,取最差输入	无假设,对所有输入成立	快排最坏 $O(n^2)$
平均情形	对输入的概率分布取平均	必须假设输入分布(如"等可能")	快排平均 $O(n \log n)$
摊还	对操作序列取平均	无概率假设,对任何序列成立	n 次 <code>push_back</code> 总代价 $O(n)$

摊还分析(amortized analysis):数据结构上任意一个由 n 个操作组成的序列,若总代价为 $O(n \cdot f(n))$,就说每个操作摊还代价为 $O(f(n))$ 。它不假设任何概率分布,对最恶劣的序列也成立。

11.1.2 为什么需要摊还分析

回头看前几章,至少有三个结论是"背下来"而没证明的:

- **第3章**:vector 倍增扩容使 `push_back` "平均 $O(1)$ "——没证明,也没说清是摊还;
- **第6章**:`buildHeap` 是 $O(n)$ ——逐个 `insert` 明明 $O(n \log n)$,整体下滤为何线性?
- **第6章**:斜堆 `merge` 摊还 $O(\log n)$ ——单个 `merge` 最坏 $O(n)$,教材直接给结论;

三个结论分别对应本章的三种方法:聚集证 `buildHeap`,会计证动态数组,势能证斜堆——学完本章,"背诵"变"推导"。摊还分析是算法设计的"定价"工具:一次操作贵没关系,整个序列便宜就行。

⚠ 误区 1:把摊还 $O(1)$ 当单次 $O(1)$

摊还 $O(1)$ 的意思是" n 次操作总代价 $O(n)$ ",**不保证单次操作是 $O(1)$** 。vector 那次触发扩容的 `push_back`,单次代价仍是 $O(n)$ ——只是被前面的便宜 `push` 平摊了。同理,"斜堆 `merge` 摊还 $O(\log n)$ "不代表某一次 `merge` 不会花 $O(n)$ 。实时系统里单次延迟上限有时更关键,这时需要第12章平衡树那种"每次最坏 $O(\log n)$ "的结构。

11.1.3 摊还代价的定义

设操作序列共 n 个操作,第 i 个操作的实际代价记为 c_i ,总代价 $T(n) = \sum_{i=1}^n c_i$ 。若对任意长度为 n 的合法序列都有 $T(n) = O(n \cdot f(n))$,则称每个操作的**摊还代价**为 $O(f(n))$ 。三种方法回答同一问题——"怎么证 $T(n)$ 的上界"——只是记账

方式不同:

方法	一句话	数学工具
聚集分析	先算总账,再平均	Σ 求和
会计方法	每个操作预存,贵的取款	账户余额非负
势能方法	把"存款"记在数据结构状态上	势能函数 Φ 的差

 记号说明

本章公式密集,记号统一: c_i 第 i 个操作的**实际代价**, a_i 其**摊还代价**; $\Phi(D_i)$ 第 i 个操作结束后结构的**势能**(读"Phi"), $\Delta\Phi = \Phi(D_i) - \Phi(D_{i-1})$ 。

§11.2 聚集分析:栈操作序列

11.2.1 方法:先求和,再平均

聚集分析最直接:把 n 个操作的实际代价**全部加起来**得总代价 $T(n)$,除以 n 即摊还代价。难点只在"证总代价上界"——通常靠巧妙的计数观察,而非逐个求和。

11.2.2 经典例子:栈的 push/pop/multiPop

考虑一个从空栈开始的栈,三个操作:push(代价 1)、pop(代价 1)、multiPop(k)(连续弹出 k 个元素,不够时弹到空,代价 = 实际弹出个数,最坏 $O(n)$)。问:任意 n 个操作,总代价是多少?

直接求和会卡住:某次 multiPop 单次就花 $O(n)$,但序列里不可能全是 multiPop——弹出的元素**必须先被 push 进去**,而 n 个操作最多 push n 次。 **关键观察:每个元素至多入栈一次、至多出栈一次**,于是:

出栈总次数 $\leq$ 入栈总次数 $\leq n$,总代价 $\leq 2n$,摊还代价 $O(1)$ 。证明没做任何输入假设: **任何** 长度为 n 的序列总代价都 $\leq 2n$ 。若问"全是 multiPop 怎么办?"——这种序列根本不存在:栈空了就没得弹。这个"不可能"正是证明核心。

11.2.3 代码验证:总代价 $\leq 2n$

示例 11-1 统计栈操作序列的实际总代价

```

#include <iostream>
#include <vector>
using namespace std;

int main() {
    vector<int> st;
    long long total = 0;
    int n = 10000;
    for (int i = 0; i < n; ++i) {
        st.push_back(i);           // push:代价 1
        total += 1;
        if (i % 1000 == 999) {     // 每 1000 次 push 来一个大 multiPop
            while (!st.empty()) {   // 弹空:每个元素只出栈一次
                st.pop_back();
                total += 1;
            }
        }
    }
    cout << "操作数 n = " << n
         << ", 总代价 = " << total
         << " ( $\leq 2n =$ " << 2 * n << ") "
         << ", 摊还 = " << (double)total / n << endl;
    return 0;
}

```

§11.3 会计方法:动态数组的"存 3 元"证明

11.3.1 方法:预存存款

会计方法把摊还代价看作"收费":每个操作收摊还代价 a_i 元,其中 c_i 元支付该操作本身, $a_i - c_i$ 元存入**账户**,留给未来昂贵操作取用。唯一要求是:

任何时刻账户余额非负:对每个前缀 $\sum_{i=1}^k (a_i - c_i) \geq 0$ 。

余额非负 $\Rightarrow$ 累加得 $\sum_{i=1}^n a_i \geq \sum_{i=1}^n c_i$,总摊还代价是总实际代价的**上界**。故每个操作收费 $O(1) \Rightarrow$ 总实际代价 $O(n)$ 。

11.3.2 动态数组:插入收 3 元

回到第 3 章的动态数组:push_back 平时只写一个元素(实际代价 1);容量满时倍增扩容要拷贝全部旧元素(实际代价 $O(n)$)。定价 **3 元**:

- 1 元:支付本次写入;
- 2 元:存入账户,留给"未来某次扩容时本元素的拷贝";

扩容时每拷贝一个旧元素从账户支付 1 元。直觉:设扩容前容量为 m (已满),拷贝费 m 元;这 m 个元素中后 $m/2$ 个是**上次**扩容后才插入的,每个带 2 元存款,合计恰好 m 元;前 $m/2$ 个"老元素"的存款原封留存。归纳下去,账户永不负。

11.3.3 存款非负的 Σ 推导

严格版本用总账。设元素个数 s 、容量 $c = 2^k$,累计扩容拷贝费是等比级数: $\sum_{j=0}^{k-1} 2^j = c - 1$ 。总收费 $3s$,总花费 s (写入) + $(c - 1)$ (拷贝),余额 $= 3s - s - (c - 1) = 2s - c + 1$ 。数组满才扩容,刚扩完 $s = c/2$,故 $s \geq c/2$,余额 $\geq 1 > 0$ 。

余额恒正 $\Rightarrow$ 总摊还 $\geq$ 总实际 $\Rightarrow$ 总代价 $\leq 3n$, 摊还 $O(1)$ 。证明只需"容量按常数因子增长", 1.5 倍、1.125 倍都行; 唯独"每次 +1"不行(摊还 $O(n)$)。

C 的 realloc 手动扩容	C++ 的 vector 自动倍增
realloc 扩容, size 变化自己记账	push_back 自动扩容, size() / capacity() 自带
扩容策略自己写, 每次 +1 就退化到 $O(n^2)$	倍增策略内建, 摊还 $O(1)$ 由标准库保证
失败返回 NULL, 旧指针要自己处理	分配失败抛 bad_alloc, 旧内存由 RAII 释放
扩容后旧指针全部失效	扩容后迭代器失效, 下标访问安全(见 §3.3.3)

11.3.4 代码验证: 账户余额永不为负

示例 11-2 会计方法视角的"存 3 元"动态数组

```
// 每次 push 收 3 元(1 元自用 + 2 元存款), 扩容拷贝从账户支付。
class BankArray {
    int* data_ = nullptr;
    size_t size_ = 0, cap_ = 0;
public:
    long long balance = 0;           // 账户余额
    long long minBalance = 0;        // 全程最低余额
    void push_back(int x) {
        balance += 3;                // 收 3 元
        balance -= 1;                // 支付本次写入
        if (size_ == cap_) {         // 容量满: 倍增
            size_t newCap = cap_ == 0 ? 1 : cap_ * 2;
            int* p = new int[newCap];
            for (size_t i = 0; i < size_; ++i) { // 每拷 1 个付 1 元
                p[i] = data_[i];
                balance -= 1;
            }
            delete[] data_;
            data_ = p;
            cap_ = newCap;
        }
        data_[size_++] = x;
        if (balance < minBalance) minBalance = balance;
    }
    ~BankArray() { delete[] data_; }
};
```

§11.4 势能方法: 势能函数与通用框架

11.4.1 方法: 势能函数 Φ

势能方法(potential method)把会计方法的"账户"从具体元素上抽象出来, 记在**数据结构的状态**上。定义势能函数 $\Phi(D)$ 为状态 D 的一个数值属性(如"栈中元素个数"); 设 D_i 为第 i 个操作结束后的状态, D_0 为初始状态, 则第 i 个操作的摊还代价定义为:

$a_i = c_i + \Phi(D_i) - \Phi(D_{i-1})$, 即 **摊还代价 = 实际代价 + 势能变化 $\Delta\Phi$** 。

两边求和,中间项错位相消(望远镜求和): $\sum_{i=1}^n a_i = \sum_{i=1}^n c_i + \Phi(D_n) - \Phi(D_0)$ 。

只要 $\Phi(D_0) = 0$ 且 $\Phi(D_i) \geq 0$ 对所有 i 成立,就有 $\sum a_i \geq \sum c_i$,总摊还是总实际代价的上界。与会计方法唯一区别:会计把存款绑在具体元素上,势能把它绑在整个结构的状态上——会计方法是势能方法的特例。

11.4.2 栈的例子重推

取 $\Phi(D)$ = 栈中元素个数, $\Phi(D_0) = 0$,且元素个数永不为负,条件满足。逐个操作算摊还代价:

- **push**:实际 1, $\Delta\Phi = +1$,摊还 = 2;
- **pop**:实际 1, $\Delta\Phi = -1$,摊还 = 0;
- **multiPop(k)**:弹出 k 个,实际 k , $\Delta\Phi = -k$,摊还 = 0;

总摊还 = $2 \times \text{push 次数} \leq 2n$,与聚集分析一致。"贵的"multiPop 摊还代价是 0——它消耗的正是 push 积累的势能。

⚠ 误区 2:把平均情形分析当摊还分析

"快排平均 $O(n \log n)$ "与"vector 插入摊还 $O(1)$ "是两回事:前者对**输入排列**取平均,前提是"输入等可能",数据分布一变结论就失效;后者对**操作序列**取平均,对任何序列都成立——攻击者精心构造"push 到满再 push"的序列,总拷贝也 $\leq 2n$ 。口诀:"随机""等可能""期望"是平均分析;"任意序列""总代价上界"是摊还分析。

11.4.3 自调整表(move-to-front):结论与直觉

自调整表:元素排成一列,每次访问后移到表头(move-to-front),频繁访问者越用越靠前。访问位置 k 的元素实际代价 $O(k)$ (要挪动 $k-1$ 个元素),但**摊还代价 $O(\log n)$** : n 次任意访问的总代价 $O(n \log n)$ 。

证明思路(从简,见教材):取势能 Φ = 表中逆序对个数。把被访问元素从位置 k 提到表头,破坏约 k 个逆序对(势能下降),新产生的逆序对有界;势能变化与移动代价大体抵消,余项用对数界住。结论:摊还 $O(\log n)$ 。自调整思想在工程中遍地开花:LRU 缓存、"最近使用的放前面"的接口设计都是亲戚。

11.4.4 势能方法的通用框架

★ 重点:势能方法四步框架

1. **选 Φ** :只依赖数据结构的状态(元素个数、逆序对数.....);
2. **验证**: $\Phi(D_0)$ 有界且 $\Phi(D_i) \geq 0$;
3. **逐个算**: $a_i = c_i + \Delta\Phi$,昂贵操作 $\Delta\Phi$ 为负、便宜操作为正;
4. **累加得界**: $\sum a_i = O(nf(n))$ 则摊还 $O(f(n))$ 。

选 Φ 的直觉: Φ 应度量"结构中蕴藏的、未来必然要做的活儿"——待弹出的元素、待拷贝的元素、待"纠正"的逆序对。消耗这些"存量"时,实际代价由势能下降支付。

⚠ 误区 3:势能函数选错,存款为负

Φ 必须非负,这是势能方法的前提。反例:栈取 $\Phi(D) = -(\text{元素个数})$, $\Phi(D_0) = 0$ 但 $\Phi(D_i) < 0$,此时 $\sum a_i < \sum c_i$,总摊还代价**低于**总实际代价,不再是上界,"总代价 $O(n)$ "直接失效。更隐蔽的错误:选 Φ 依赖操作历史(如" Φ 取 i 的奇偶"),望远镜求和不再成立。选好 Φ 后务必检查"非负"与"只依赖状态"两条。

§11.5 摊还在堆上的应用

11.5.1 斜堆:merge 摊还 $O(\log n)$

第6章讲过斜堆:没有平衡信息,merge时无条件交换左右子堆,单次merge最坏 $O(n)$,但merge摊还 $O(\log n)$ 。证明用势能法(从简):称"重"结点为右子堆大小 $\geq$ 左子堆一半者,势能 Φ = 重结点总数。merge的实际代价 = 右路径步数 = 轻边数 + 重边数;轻边数 $\leq \log_2 n$ (每经一条轻边子树至少减半),每条重边由势能下降1支付,交换后至多产生 $\log n$ 个新重结点,两抵得 $O(\log n)$ 。斜堆是"摊还换来简单"的典型:无平衡信息、代码最短。

11.5.2 二项队列:insert 摊还 $O(1)$

二项队列的insert可看作"二进制加法":二项树大小是2的幂,合并两个同大小树就像进位 $1+1=10$,于是借用二进制计数器的经典结论:从0加1共 n 次,总翻转位数 $\leq 2n$,摊还 $O(1)$ 。故insert摊还 $O(1)$ 、最坏 $O(\log n)$;deleteMin与merge摊还 $O(\log n)$ 。对比二叉堆:insert最坏就是 $O(\log n)$,不必摊还——摊还分析只在结构把贵操作推迟分摊时,才比最坏更紧。

11.5.3 二叉堆 buildHeap:聚集分析回顾

第6章说buildHeap是 $O(n)$,现在证明。buildHeap从第 $\lfloor n/2 \rfloor$ 个结点起逐个下滤,关键是把结点按高度而非按深度计数:完全二叉树中高度为 h 的结点至多有 $n/2^{h+1}$ 个,每个下滤 $\leq h$ 层,总代价:

$$T(n) \leq \sum_{h=0}^{\log n} (n/2^{h+1}) \cdot h = (n/2) \cdot \sum_{h=1}^{\infty} h/2^h = (n/2) \cdot 2 = O(n).$$

级数 $\sum_{h=1}^{\infty} h/2^h = 2$ 。这就是"逐个insert是 $O(n \log n)$ 、整体下滤只要 $O(n)$ "的原因:多数结点在底层、下滤路径短;高层结点下滤深,但数量指数级地少。下面代码统计真实步数验证。

示例 11-3 统计 buildHeap 的总下滤步数(验证 $\approx n$)

```

// 从第 n/2 个结点向前逐个下滤,统计总步数
#include <iostream>
#include <vector>
using namespace std;

long long heapify(vector<int>& a) {
    int n = int(a.size());
    long long steps = 0;
    auto down = [&](int i) {          // 下滤
        while (true) {
            int l = 2 * i + 1, r = 2 * i + 2, m = i;
            if (l < n && a[l] > a[m]) m = l;
            if (r < n && a[r] > a[m]) m = r;
            if (m == i) break;
            swap(a[i], a[m]);
            ++steps;                    // 交换一层记 1 步
            i = m;
        }
    };
    for (int i = n / 2 - 1; i >= 0; --i) down(i);
    return steps;
}

int main() {
    int n = 65535;
    vector<int> a(n);
    for (int i = 0; i < n; ++i) a[i] = i + 1;    // 升序:对最大堆最不利
    long long s = heapify(a);
    cout << "n = " << n << ", 总下滤步数 = " << s
        << " (理论 O(n),实测 ≈ " << s * 100 / n << "% n)" << endl;
    return 0;
}
// 输出:n = 65535, 总下滤步数 = 65519 (理论 O(n),实测 ≈ 99% n)

```

§11.6 工程印证:实测扩容拐点

11.6.1 手写倍增 vs 标准库:观察扩容拐点

理论说"容量翻倍", `vector` 真的翻倍吗?打印每次扩容的 (size, capacity) 拐点即可见分晓。下面把 手写实现与标准库写法并排对照。

示例 11-4 手写倍增与 `std::vector` 并排对照:观察 capacity 拐点

```

// 手写倍增与 std::vector 并排:打印扩容拐点(size → capacity)
#include <iostream>
#include <vector>
using namespace std;

class MyVector {                                // 手写倍增动态数组
    int* data_ = nullptr; size_t size_ = 0, cap_ = 0;
public:
    void push_back(int x) {
        if (size_ == cap_) {
            size_t newCap = cap_ == 0 ? 1 : cap_ * 2;    // 倍增
            int* p = new int[newCap];
            for (size_t i = 0; i < size_; ++i) p[i] = data_[i];
            delete[] data_;
            data_ = p;
            cap_ = newCap;
            cout << size_ << "→" << cap_ << " ";    // 打印拐点
        }
        data_[size_++] = x;
    }
    ~MyVector() { delete[] data_; }
};

int main() {
    cout << "手写倍增: ";
    MyVector v;
    for (int i = 0; i < 1024; ++i) v.push_back(i);
    cout << "\nstd::vector: ";
    vector<int> sv;                                // 标准库:同样的拐点观察
    size_t prev = 0;
    for (int i = 0; i < 1024; ++i) {
        sv.push_back(i);
        if (sv.capacity() != prev) {    // 容量变化 = 扩容拐点
            cout << sv.size() << "→" << sv.capacity() << " ";
            prev = sv.capacity();
        }
    }
    cout << endl;
    return 0;
}

```

C 的 malloc 数组	C++ 的手写倍增类 / vector
没有 size/capacity 概念,长度、容量全靠变量自管	size() 与 capacity() 是内建接口,拐点一目了然
扩容写错(如每次 +1)代价是 $O(n^2)$,且只有运行慢时才发现	倍增由库保证,摊还 $O(1)$;手写类也要照抄倍增骨架
free 忘记调用即泄漏	析构函数自动释放(RAII)

11.6.2 实测数据解读

本机实测(g++ 13,libstdc++):vector 连续 push_back 1024 次,拐点序列 1→1, 2→2, 3→4, 5→8, 9→16, ..., 513→1024,从 2 起每次翻倍。核账:总拷贝 = 1+2+4+...+512 = 1023 ≈ n,共 10 次扩容。各实现初始容量常数不同,但"几何增长"的摊还结论完全相同。

🚀 工程建议:管理扩容的四个招式

- **预判容量:**知道要装多少时先 `v.reserve(n)`,把扩容次数降为 0——摊还分析告诉你"扩容不贵",但一次都别扩更便宜;
- **只读不扩:**循环里先 `reserve` 再填,迭代器失效问题就此消失;
- **缩容要谨慎:** `shrink_to_fit` 释放多余容量,但下次插入又要重新扩容——"缩了又扩"是双倍浪费,只适合"装完就长期只读"的场景;
- **增长因子不是越大越好:**2 倍省拷贝但峰值内存达 2 倍;1.5 倍更省内存、对缓存更友好。只要因子 > 1,摊还就仍是 $O(1)$ 。

C 的手写扩容代码	C++ 的标准库接口
<code>realloc</code> 每次扩容都是裸内存搬运,策略(倍增还是 +1)写死在调用点	<code>push_back</code> 自动倍增;策略由实现决定, <code>capacity()</code> 把拐点暴露成可观测接口
想预分配要自己写"capacity 变量 + 判断"样板	<code>reserve(n)</code> 一行预分配,扩容次数归零
<code>realloc</code> 返回 NULL 要检查,失败后旧指针仍有效	分配失败抛 <code>bad_alloc</code> ,异常安全由标准库保证

⚠ 误区 4:扩容增量太小,摊还证明失效

"每次 +1"的扩容不是常数因子增长:第 i 次扩容拷贝 $i-1$ 个元素,总拷贝 $\sum_{i=1}^n i \approx n^2/2$,摊还 $O(n)$ ——插入 10 万元素就要拷约 50 亿次。vector 扩容系数是 2,Python 的 list 约 1.125,都是几何增长;"每次多开固定格数"的扩容是平方级灾难。写 C 的 `realloc` 时这是最常见的性能坑:功能正确、规模一大就慢十倍百倍,只靠计时才能发现。

§11.7 三种方法的选用总结

11.7.1 一张表看清三种方法

方法	核心思路	优点	局限	经典用例
聚集分析	总代价求和再平均	最直接	整体求和要可做	栈序列、buildHeap、二进制计数器
会计方法	每操作预存、贵的取款	最直观	要设计收费方案	动态数组"存 3 元"、二项队列 insert
势能方法	势能函数挂在状态上	最强大	选 Φ 靠经验	斜堆 merge、自调整表、splay 树

11.7.2 选型建议

三个方法算的是同一笔账,结论互相印证。选型:结构简单、总量可数,用聚集分析(栈、buildHeap);要给每个操作"讲明白为什么便宜",用会计方法(动态数组);结构复杂、代价藏在状态里,用势能方法(斜堆、自调整表)。摊还分析不是课堂玩具:数据库批量提交、C++ 的 vector、Python 的 list,全是"贵操作由便宜操作平摊"的生意经。

本章复杂度速查表

操作	数据结构 / 算法	平均	最坏
尾部插入 push_back	动态数组(倍增扩容)	摊还 $O(1)$	$O(n)$ (扩容那次)
尾部插入	动态数组(每次 +1 扩容)	$O(n)$	$O(n)$
push	栈(数组或链表)	$O(1)$	$O(1)$
pop	栈	$O(1)$	$O(1)$
multiPop(k)	栈	摊还 $O(1)$	$O(n)$ (单次)
n 次 push/pop/multiPop 序列	栈(聚集分析)	总代价 $O(n)$, 每个元素至多入出栈一次	
插入(存 3 元)	动态数组(会计方法)	总摊还 $\leq 3n$, 账户余额恒 ≥ 1	
加 1 / insert	二进制计数器 / 二项队列	摊还 $O(1)$	$O(\log n)$ (单次)
访问(移到表头)	自调整表(move-to-front)	摊还 $O(\log n)$	$O(n)$
insert	二叉堆	$O(\log n)$	$O(\log n)$
buildHeap	二叉堆(聚集分析)	$O(n)$	$O(n)$
merge	斜堆(势能方法)	摊还 $O(\log n)$	$O(n)$
merge / deleteMin	二项队列	摊还 $O(\log n)$	$O(\log n)$
会计方法	通用框架	余额非负 $\Rightarrow$ 总摊还 $\geq$ 总实际	
势能方法	通用框架	$a_i = c_i + \Delta\Phi$, Φ 非负 $\Rightarrow$ 摊还为上界	

最小必会清单

- 能独立说出摊还、平均、最坏三种分析的区别,并各举一例
- 能独立证明:任意 n 个 push/pop/multiPop 操作总代价 $\leq 2n$
- 能独立默写"插入收 3 元"的会计证明,并推导余额 $= 2s - c + 1 \geq 1$
- 能独立解释"每次 +1"扩容为何 $O(n^2)$,几何扩容为何摊还 $O(1)$
- 能独立写出势能定义式 $a_i = c_i + \Delta\Phi$,并解释 Φ 为何必须非负
- 能用势能方法(Φ = 元素个数)重推栈:push 2、pop 0、multiPop 0
- 能独立用聚集分析证明 buildHeap 是 $O(n)$ (按高度计数)
- 能独立说出斜堆 merge 摊还 $O(\log n)$ 、二项队列 insert 摊还 $O(1)$

自测题

Q 1. 摊还分析与平均情形分析的根本区别是什么?

✓ 参考答案

平均情形分析对**输入**的分布取平均(如"快排平均 $O(n \log n)$ "假设排列等可能),分布一变结论即失效;摊还分析对**操作序列**取平均,不假设任何概率——" n 次 push_back 总代价 $O(n)$ "对任何序列都成立。

Q 2. 为什么说"全 multiPop 的序列"不存在?这对证明意味着什么?

✓ 参考答案

multiPop 要弹出元素,而元素必须先 push 进去;从空栈开始的序列里,弹出总次数 $\leq$ 入栈总次数 $\leq n$,multiPop 的总代价被 push 次数封顶——昂贵操作在结构上不可能频繁出现,总代价 $\leq 2n$ 对任何序列成立。

Q 3. 动态数组扩容"存 1 元"为什么不够?给出会破产的序列。

✓ 参考答案

存 1 元 = 每次插入收费 2(1 元自用 + 1 元存款)。设 size = s 、容量 $c = 2^k$,总收费 $2s$,总花费 $s + (c - 1)$,余额 = $s - c + 1$ 。数组一满($s = c$)时余额只剩 1;此时扩容:收 2 元、付 1 元自用、再付 c 元拷贝,余额 = $1 + 2 - 1 - c = 2 - c < 0$ —— $c \geq 4$ 即破产。而"存 2 元"(收 3 元)时余额 = $2s - c + 1$,利用 $s \geq c/2$ 得余额 ≥ 1 恒正。扩容是一次性大额开销,存款必须按"每个元素都积累足够配额"定价,2 元是最低安全价。

Q 4. buildHeap 为什么是 $O(n)$ 而不是 $O(n \log n)$?差在哪一步?

✓ 参考答案

逐个 insert 到空堆是 $O(n \log n)$,因为每个元素都要从底部上滤 $\log n$ 层;buildHeap 整体下滤时,多数结点在底层、下滤路径短:高度 h 的结点只有 $n/2^{h+1}$ 个,每个下滤 $\leq h$ 层,总代价 $(n/2) \cdot \sum h/2^h = O(n)$ 。差在"按高度计数"而非"按深度计数",同一棵树的两种视角。

Q 5. 为什么 vector 扩容后迭代器全部失效,而 std::list 不会?这与摊还分析有何联系?

✓ 参考答案

vector 扩容要整体搬家到新内存,旧地址上的一切指针/迭代器随之作废——这是"倍增拷贝"的直接后果,也是摊还 $O(1)$ 付出的代价(总拷贝 $\approx 2n$ 由摊还证明兜底);list 的每个结点独立分配,插入/删除只改指针不搬数据,迭代器天然稳定,代价是按索引访问 $O(n)$ 。选型时,"迭代器稳定"是链表唯一的决胜点。

章末实验题:动态数组扩容的摊还代价实测

实验 11:动态数组扩容的摊还代价实测

实验目标:手写带拷贝计数器的动态数组,用真实数据验证"倍增扩容总拷贝 $\approx 2n$ 、每次 +1 扩容总拷贝 $\approx n^2/2$ ",并用会计方法解释"存 3 元"为什么够。

实验要求:

- 任务 1:手写倍增动态数组 DoublingArray,在扩容处累计真实拷贝次数,提供 totalCopies/lastCost/maxCost 三个统计量(知识点:\$11.3 会计方法、第 3 章倍增扩容)
- 任务 2:连续插入 $n = 10000$ 个元素,统计总拷贝,验证倍增 $\approx 2n$ vs 每次 +1 的笨扩容 $\approx n^2/2$ (知识点:\$11.2)
- 任务 3:对比两种策略的"单次最大代价"与"平均代价",解释摊还 $O(1)$ 与"单次最坏 $O(n)$ "并存(知识点:\$11.1)
- 任务 4:用会计方法注释 BankArray 收费规则,验证全程最低余额 ≥ 0 ,写出余额公式 $2s - c + 1$ (知识点:\$11.3)
- 任务 5:观察标准库 vector 的 capacity 拐点序列,与手写倍增对照(知识点:\$11.6)

实验环境:C++17,需 g++ 或 MSVC。编译: `g++ -std=c++17 -O2 -o lab11 lab11.cpp`;运行 lab11 (Windows 为 lab11.exe)。

第一步 个类共用同一骨架:扩容时先分配新数组,循环拷贝旧元素并更新计数——拷贝计数必须放在真实 for 循环里,不能用 `memcpy` 或 `std::copy`,否则统计失真

第二步 扩容 $n = 10000$ 时总拷贝约 5000 万次,int 会溢出,计数器一律用 long long

第三步 任务 4 的余额更新放在 push_back 末尾统一做,并记录全程最低余额;理论下界 1,实测应 ≥ 1

第四步 任务 5:记录上一次 capacity(),每次 push 后比较,变化即打印 size→capacity

✅ 参考答案(完整可运行程序,分三段)

```
// lab11.cpp — 实验 11:动态数组扩容的摊还代价实测(第一段:两种策略)
#include <iostream>
#include <vector>
using namespace std;

// 任务 1:带真实拷贝计数器的倍增动态数组
class DoublingArray {
    int* data_ = nullptr;
    size_t size_ = 0, cap_ = 0;
public:
    long long totalCopies = 0, lastCost = 0, maxCost = 0;
    void push_back(int x) {
        lastCost = 0;
        if (size_ == cap_) { // 满:倍增扩容
            size_t newCap = cap_ == 0 ? 1 : cap_ * 2;
            int* p = new int[newCap];
            for (size_t i = 0; i < size_; ++i) p[i] = data_[i];
            lastCost = (long long)size_; // 本次 push 的拷贝代价
            totalCopies += size_;
            if (lastCost > maxCost) maxCost = lastCost;
            delete[] data_;
            data_ = p;
            cap_ = newCap;
        }
        data_[size_++] = x;
    }
    ~DoublingArray() { delete[] data_; }
};

// 任务 1:每次只多 1 格的"笨扩容"(对照策略)
class LinearArray {
    int* data_ = nullptr;
    size_t size_ = 0, cap_ = 0;
public:
    long long totalCopies = 0, lastCost = 0, maxCost = 0;
    void push_back(int x) {
        lastCost = 0;
        if (size_ == cap_) { // 满:只多开 1 格
            int* p = new int[cap_ + 1];
            for (size_t i = 0; i < size_; ++i) p[i] = data_[i];
            lastCost = (long long)size_;
            totalCopies += size_;
            if (lastCost > maxCost) maxCost = lastCost;
            delete[] data_;
            data_ = p;
            ++cap_;
        }
        data_[size_++] = x;
    }
    ~LinearArray() { delete[] data_; }
};
```

lab11.cpp(第二段:会计方法"存 3 元"动态数组)

```
// 任务 4:每次 push 收 3 元(1 元自用 + 2 元存款),扩容每拷 1 个付 1 元。
// 余额 = 总收费 3s - 总花费 (s + c - 1) = 2s - c + 1 ≥ 1,恒正 → 摊还 O(1)。
class BankArray {
    int* data_ = nullptr;
    size_t size_ = 0, cap_ = 0;
public:
    long long balance = 0;           // 账户余额
    long long minBalance = 1LL << 60; // 全程最低余额(应恒 ≥ 0)
    void push_back(int x) {
        balance += 3;                // 收 3 元
        balance -= 1;                // 支付本次写入
        if (size_ == cap_) {         // 扩容:每拷 1 个元素付 1 元
            size_t newCap = cap_ == 0 ? 1 : cap_ * 2;
            int* p = new int[newCap];
            for (size_t i = 0; i < size_; ++i) {
                p[i] = data_[i];
                balance -= 1;
            }
            delete[] data_;
            data_ = p;
            cap_ = newCap;
        }
        data_[size_++] = x;
        if (balance < minBalance) minBalance = balance;
    }
    ~BankArray() { delete[] data_; }
};
```

lab11.cpp(第三段:主程序)

```

int main() {
    const int N = 10000;    // 连续插入的元素个数

    // ---- 任务 2、3:拷贝统计与代价对比 ----
    DoublingArray d;
    for (int i = 0; i < N; ++i) d.push_back(i);

    cout << "[任务2/3] 倍增策略:连续插入 " << N << " 个元素\n"
        << " 总拷贝次数 = " << d.totalCopies
        << " (理论  $\approx 2n =$  " << 2 * N << ") \n"
        << " 单次最大代价 = " << d.maxCost
        << " 次拷贝(仅发生在扩容的那一次 push) \n"
        << " 平均代价 = " << (double)d.totalCopies / N
        << " 次拷贝/push \n\n";

    LinearArray l;
    for (int i = 0; i < N; ++i) l.push_back(i);
    cout << "[任务2/3] 笨扩容(每次 +1):连续插入 " << N << " 个元素\n"
        << " 总拷贝次数 = " << l.totalCopies
        << " (理论  $\approx n^2/2 =$  " << (long long)N * N / 2 << ") \n"
        << " 单次最大代价 = " << l.maxCost
        << " 次拷贝(最后一次扩容) \n"
        << " 平均代价 = " << (double)l.totalCopies / N
        << " 次拷贝/push \n\n";

    // ---- 任务 4:验证"存 3 元"够不够 ----
    BankArray b;
    for (int i = 0; i < N; ++i) b.push_back(i);
    cout << "[任务4] 会计方法:存 3 元策略运行 " << N << " 次 push 后\n"
        << " 账户余额 = " << b.balance
        << ", 全程最低余额 = " << b.minBalance
        << " (非负即证明有效) \n\n";

    // ---- 任务 5:vector 的 capacity 拐点 ----
    vector<int> v;
    size_t prev = 0;
    cout << "[任务5] std::vector 扩容拐点(size  $\rightarrow$  capacity): \n ";
    for (int i = 1; i <= 2048; ++i) {
        v.push_back(i);
        if (v.capacity() != prev) {          // 容量变化 = 一次扩容拐点
            cout << v.size() << "  $\rightarrow$  " << v.capacity() << " ";
            prev = v.capacity();
        }
    }
    cout << "\n\n结论: vector 采用倍增策略, 拐点序列 1,2,4,8,..., "
        << "总拷贝  $\approx 2n$ , 与本章理论完全一致。" << endl;
    return 0;
}

```

编译与运行,以及本机实测输出:

```
g++ -std=c++17 -O2 -o lab11 lab11.cpp
./lab11
```

[任务2/3] 倍增策略:连续插入 10000 个元素

总拷贝次数 = 16383 (理论 $\approx 2n = 20000$)

单次最大代价 = 8192 次拷贝(仅发生在扩容的那一次 push)

平均代价 = 1.6383 次拷贝/push

[任务2/3] 笨扩容(每次 +1):连续插入 10000 个元素

总拷贝次数 = 49995000 (理论 $\approx n^2/2 = 50000000$)

单次最大代价 = 9999 次拷贝(最后一次扩容)

平均代价 = 4999.5 次拷贝/push

[任务4] 会计方法:存 3 元策略运行 10000 次 push 后

账户余额 = 3617, 全程最低余额 = 2 (非负即证明有效)

[任务5] `std::vector` 扩容拐点(size $\rightarrow$ capacity):

1 $\rightarrow$ 1 2 $\rightarrow$ 2 3 $\rightarrow$ 4 5 $\rightarrow$ 8 9 $\rightarrow$ 16 17 $\rightarrow$ 32 33 $\rightarrow$ 64 65 $\rightarrow$ 128 129 $\rightarrow$ 256 257 $\rightarrow$ 512 513 $\rightarrow$ 1024 1025 $\rightarrow$ 2048

结论: `vector` 采用倍增策略,拐点序列 1,2,4,8,...,总拷贝 $\approx 2n$,与本章理论完全一致。

设计说明:① 三个类共用"满则扩容、循环拷贝"的骨架,差异只在扩容因子($\times 2$ 与 $+1$),暴露摊还分析的唯一变量;② 拷贝计数放在真实 for 循环而非 `memcpy`,统计语义代价而非内存搬运字节;③ 任务 4 用可运行的"余额"把抽象证明变成可观察的量,实测最低余额 2 与理论下界 1 吻合;④ 任务 5 对照标准库 `vector`,把"平均 $O(1)$ "落到实地:拐点正是 §11.3 证明的倍增策略。

下一章预告:第12章 高级数据结构——红黑树、Treap、跳表、k-d 树;以及标准库选择指南。本章的势能方法是第12章伸展树(splay tree)摊还分析的直接武器;红黑树的摊还/最坏界限对比、Treap 的随机化分析,都会用到本章的三种方法。